

Application Layer Latency

Day 2: Network and OS Foundations for Safe
Robotic Actuation

Mentor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

July 2026

Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

Table of content

1. From One Wire to Many Talkers
2. MQTT and the Topic Graph
3. QoS: Three Delivery Contracts
4. The End-to-End Latency Budget
5. Project Hands-on Preview: Build the Bridge

Yesterday and Today

Yesterday: one wire, two parties.

You built a frame format with CRC-16 and proved one thing: the arm never moves on a corrupted frame.

Today: many talkers, one arm.

An operator console, a planner, an e-stop button and a logger all want to reach that arm. Who connects to whom, and what does the answer cost in milliseconds?

The invariant has not changed.

Correctly, and in time. Today adds the layer where *in time* is won or lost: the messaging layer between processes.



From One Wire to Many Talkers

More Than One Sender

The cast around our arm.

Operator console (commands), motion planner (command streams), e-stop button (one message that beats all others), dashboard (wants to see everything).

Point-to-point does not scale here.

Direct connections between every pair of programs gives a tangle that grows with every new tool, and every sender must know every receiver's address.

The question of the morning.

What should sit in the middle, and what does the middleman cost on the path of an emergency stop?

Model 1: Request/Response

The shape.

A client opens a connection to a known server, sends a request, and *waits* for the reply. HTTP is the famous example.

What it couples.

Identity: the client must know exactly whom to ask. Time: both sides must be up at the same moment. One question, one answer, one pair.

Where it shines.

Queries with exactly one authoritative answerer: fetch a configuration, ask a database, call an API.

Model 2: Publish/Subscribe

The shape.

Producers *publish* messages to a named *topic* on a broker. Consumers *subscribe* to topics. Publisher and subscriber never address each other.

What it decouples.

Space: senders do not know receivers. Time: the broker holds the connection state. Cardinality: one message fans out to any number of subscribers.

Where it shines.

Events that several parties care about: sensor readings, state changes, and a stop command that *everyone* must hear at once.

Model 3: Peer-to-Peer

The shape.

No permanent server at all: end systems talk to each other directly, and every peer both requests and provides service. New peers bring new capacity.

Where it shines.

Bulk distribution among equals, such as file sharing. You met this architecture in Kurose Chapter 2 alongside client-server.

Why not for an actuator.

An arm is not a peer among equals. Safety wants *one* authoritative path to the hardware, with a gatekeeper on it; intermittently connected peers talking any-to-any is the exact opposite of that.

Choosing for a Robot System

Commands and telemetry are events.

Many producers, several consumers, fan-out on emergencies. This is pub-sub territory, and it is what real robot platforms use for their internal bus.

The price of the middleman.

Every message now takes two network hops (publisher to broker, broker to subscriber) plus a queue inside the broker. Latency becomes a *design* question.

Today's tool.

MQTT, the most widely deployed lightweight pub-sub protocol, and Mosquitto, its standard open-source broker and C client library.



MQTT and the Topic Graph

MQTT in One Frame

What it is.

A thin publish/subscribe protocol over TCP, designed for small devices and unreliable networks. A broker routes messages; clients publish and subscribe.

Why it fits actuation.

Tiny packet overhead (2-byte fixed header), built-in delivery guarantees (QoS), a keep-alive heartbeat, and a death notice (the last will) for clients that vanish.

Our setup.

One Mosquitto broker on the lab network; your bridge is a C client using `libmosquitto`; every laptop in the room is a potential publisher.

Topics: the Second Contract of the Week

Topics are hierarchical names.

Levels separated by /, no registration needed: publishing to a topic creates it. Subscriptions may use wildcards: + for one level, # for a subtree.

Topic	Carries	Direction
arm/cmd	framed commands (Day 1 format)	tools → bridge
arm/estop	emergency stop	anyone → bridge
arm/ack	ACK/NACK results	bridge → tools
arm/state	current arm pose	bridge → dashboards

The topic graph is a contract, like your frame spec.

The HCR platform keeps one canonical topic table that every component obeys; changing it requires a review, not a quick edit. Treat this table the same way.

Retained Messages and the Last Will

Retained: the broker remembers one message per topic.

Publish `arm/state` as retained, and a dashboard that subscribes a minute later *immediately* receives the current pose instead of staring at nothing.

Last will: a death notice.

At connect time the bridge registers a will message. If it dies without saying goodbye, the broker publishes the will for it, and every subscriber learns the path to the arm is gone.

Safety reading.

Retained state answers *what is the arm doing now*; the will answers *is anyone still driving it*. Both are one flag in the publish call.

Last Will, in More Detail

The death-of-the-initiator problem.

The bridge is the one program holding the live serial fd. If it crashes, every dashboard and tool on the bus is talking to thin air, but their connections to Mosquitto are still open.

What LWT does.

At CONNECT time the bridge registers a will. On orderly DISCONNECT the broker drops it; on missed keep-alive, the broker *publishes the will for the bridge*.

Why this is a safety primitive.

The HCR platform calls this *initiator-disconnect detection*: every subscriber learns within one keep-alive that the arm is unmanaged, dashboards mark it offline, tools refuse to publish new commands.

The Conversation on the Wire

Every MQTT session is a small set of paired packets:

- CONNECT / CONNACK: open a session, present client ID and will.
- SUBSCRIBE / SUBACK: register a topic filter, get granted QoS back.
- PUBLISH (+ its QoS handshake): the actual payload.
- PINGREQ / PINGRESP: keep-alive when nothing else flows.
- DISCONNECT: orderly goodbye; anything else triggers the will.

Where latency hides.

Connection setup costs a TCP handshake plus CONNECT/CONNACK before the first publish. Keep connections open; do not connect per message.

What a Fresh Connection Costs

The HTTP lesson, from Kurose Chapter 2.

Non-persistent HTTP pays two round trips per object: one for the TCP handshake, one for request and response. Persistent HTTP pays the setup once and reuses the pipe.

The same arithmetic in MQTT.

A fresh client pays the TCP handshake plus CONNECT/CONNACK before its first PUBLISH can leave. Connecting per message triples the latency of every command.

The rule.

Connect once, stay connected, let keep-alive hold the pipe open. On our LAN this is the difference between a sub-millisecond e-stop and a several-millisecond one.

The Bridge in C: Connect and Subscribe

A complete client in one screen.

libmosquitto drives the whole conversation from the previous frame.

```
1 int main(void)
2 {
3     mosquitto_lib_init();
4     struct mosquitto *m = mosquitto_new("bridge-01", true, NULL);
5     mosquitto_message_callback_set(m, on_message);
6     mosquitto_connect(m, "localhost", 1883, 60);
7     mosquitto_subscribe(m, NULL, "arm/cmd", 1);
8     mosquitto_loop_forever(m, -1, 1); /* blocks here */
9     return 0;
10 }
```

The Bridge in C: Connect and Subscribe (Cont'd)

The message callback is where your Day 1 code returns.

The bridge refuses to forward damage; validation runs *before* the serial port.

```
1 void on_message(struct mosquitto *m, void *userdata,  
2               const struct mosquitto_message *msg)  
3 {  
4     if (frame_validate(msg->payload, msg->payloadlen))  
5         serial_write(serial_fd, msg->payload, msg->payloadlen);  
6     else  
7         mosquitto_publish(m, NULL, "arm/ack", 4, "NACK", 1, false);  
8 }
```

One warning.

`mosquitto_loop_forever` owns the thread. Day 5 replaces it with an event loop you control.



QoS: Three Delivery Contracts

What Each Topic Demands

Requirements first, mechanism second.

Kurose Chapter 2 classifies applications by loss tolerance and time sensitivity. The same table, written for our topics, is the requirement sheet that QoS must satisfy.

Topic	Data loss	Timing	Rate
arm/cmd	no loss	bounded delay	bursts
arm/estop	no loss	hard deadline	rare
arm/ack	no loss	bounded delay	mirrors cmd
arm/state	loss-tolerant	fresh beats complete	steady 50 Hz

Keep this table in sight.

The rest of this section is a menu of delivery mechanisms. A mechanism is only right if it meets the row it serves.

Does TCP Not Already Guarantee Delivery?

What TCP guarantees.

Bytes arrive intact and in order *between two living sockets*. Yesterday's CRC story, solved one layer down.

What TCP cannot see.

A client that disconnects, crashes, or reconnects mid-message. The bytes died with the socket; TCP has nothing to retransmit and nobody to retransmit to.

QoS is delivery insurance across sessions.

MQTT adds its own acknowledgement layer on top of TCP, so a message can survive the death of the connection that first carried it.

QoS 0: At Most Once

The contract.

One PUBLISH packet, no acknowledgement, no retry. Fire and forget.

Cost and risk.

Cheapest possible: one flight, no state. If the connection drops at the wrong moment, the message is simply gone, and nobody is told.

Right for.

High-rate telemetry where the next sample supersedes the lost one. arm/state at 50 Hz does not mourn one missing pose.

QoS 1: At Least Once

The contract.

PUBLISH is answered by PUBACK. No PUBACK before the timeout: sender retransmits, flagged as a duplicate.

Cost and risk.

Two flights per message plus retry state. The message cannot be silently lost, but the receiver may see it *twice* when a PUBACK goes missing.

Sound familiar?

This is exactly yesterday's ACK/NACK-and-retransmit pattern, one layer up. And the same cure applies: your frame's SEQ field makes duplicates detectable.

QoS 2: Exactly Once

The contract.

A four-packet handshake: PUBLISH, PUBREC, PUBREL, PUBCOMP. Both sides track the message ID until the dance completes.

Cost and risk.

Four flights, two round trips, state on both ends. No loss, no duplicates, and the highest latency of the three. Under load, the broker works hardest here.

Right for.

Messages that are neither idempotent nor recoverable: a billing event, a once-only configuration change. Rarely the right answer for a motion stream.

The QoS Bill, Side by Side

QoS	Packets	Round trips	Can drop?	Can duplicate?
0 (at most once)	1	0	yes	no
1 (at least once)	2	1	no	yes
2 (exactly once)	4	2	no	no

Read the table as a menu of trade-offs.

You buy delivery certainty with round trips, and round trips are latency. In the hands-on hour you will put real numbers from our lab network into this table.

One subtlety.

QoS is negotiated per subscription too: a QoS 2 publish to a QoS 0 subscriber is delivered with QoS 0 effort. The weakest leg sets the guarantee.

Which QoS for Which Topic

arm/state: QoS 0.

Fresh beats complete. A lost pose is replaced 20 ms later by a fresher one.

arm/cmd: QoS 1 plus your SEQ field.

Loss is unacceptable; duplicates are detectable for free with yesterday's SEQ. Cheaper than QoS 2, just as safe for us.

arm/estop: QoS 1.

A duplicated stop is harmless (stopping twice is stopping); a lost stop is catastrophic. At-least-once is exactly the right contract.

The platform lesson.

The HCR platform specifies delivery semantics *per topic*, never as one global QoS number.

The Five Conformance Examples

From the HCR project: five canonical fixtures.

Every bridge, simulator or real, must produce the same behaviour on these. They are the executable definition of correctness.

Fixture	What the bridge must do
Happy path	valid frame in, ACK out, arm moves
Skin-distance violation	sandbox rejects, NACK, arm holds
Ambiguous command	ask for clarification, no action
Out-of-order arrival	detect SEQ gap, reorder or reject
Sensor impairment	enter safe state, stop accepting motion

The reading.

Three of these (*out-of-order*, *ambiguous*, *impairment*) are about refusing to act. Day 1's CRC, today's QoS, Friday's gatekeeper all serve the same goal.

In-Class Quiz: the Duplicate Arm

Your bridge subscribes to `arm/cmd`. Which QoS level can deliver the *same command twice* to the bridge?

1. QoS 0
2. QoS 1
3. QoS 2
4. None of them

In-Class Quiz: Answer

Answer. 2

Why.

QoS 1 retransmits whenever a PUBACK is lost, so the subscriber can receive the message again.

QoS 0 may drop but never repeats; QoS 2's handshake filters repeats out.

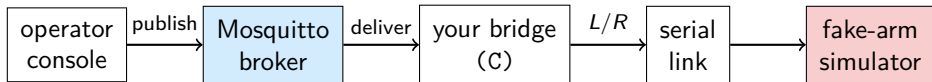
Why it matters here.

A duplicated MOVE moves the arm twice. That is why `arm/cmd` rides QoS 1 *only* in combination with the SEQ field you designed yesterday.



The End-to-End Latency Budget

The Path of One Command



Each arrow is a delay you now understand.

Publish and deliver each cost a network hop plus QoS handshake flights. The serial hop is yesterday's L/R . Queues can form at every box.

The end-to-end number is what safety cares about.

No single layer sees the whole path. Only a measurement from publish to actuation does.

What Each QoS Level Adds to the Path

QoS 0.

The floor: one broker hop each side, nothing to wait for.

QoS 1.

One extra round trip per leg. Sub-millisecond on a quiet LAN; on a congested one it queues like everything else.

QoS 2.

Two extra round trips per leg, plus broker bookkeeping on both ends.

Today's hands-on measurement.

The same arm/estop round trip at QoS 0, 1 and 2: median and worst case. Keep the distribution, not just the mean.

The Retransmission Tax

Where retries come from.

A missing PUBACK triggers retransmission after a timeout. The timeout is a policy you choose, not a law of nature.

The tension.

Too short: spurious retransmits, duplicate floods, wasted bandwidth. Too long: a genuinely lost e-stop waits the full timeout before its second chance.

Safety reading.

For arm/estop, the retry timer is part of the safety budget. If policy demands a stop within 50 ms, a 1-second retransmission timeout means the guarantee comes from the *first* delivery, and the retry is only a backstop.

The Budget Table, One Day Later

Stage	Bound	Source
Publish → broker → bridge	measured today, per QoS	Day 2 lab
Bridge validate + forward	measured Day 3	OS, sockets
Serial transmission	$L/R \approx 1.39 \text{ ms (16 B)}$	Day 1
Simulator parse → halt	measured Day 5	epoll e-stop

Yesterday one row was exact; today a second row becomes measurable.

By Friday every row has a number, and the sum either fits inside 50 ms or it does not. That sum is the engineering answer to *is this arm safe to stand next to*.

Timing a Round Trip in C

One clock, both ends.

The payload carries the start time; the echo handler reads the same monotonic clock.

```
1 struct timespec t0, t1;
2 clock_gettime(CLOCK_MONOTONIC, &t0);
3 mosquitto_publish(m, NULL, "arm/estop", sizeof t0, &t0, qos, false);
4
5 /* later, in on_message() for the echo topic: */
6 clock_gettime(CLOCK_MONOTONIC, &t1);
7 double ms = (t1.tv_sec - t0.tv_sec) * 1e3
8             + (t1.tv_nsec - t0.tv_nsec) / 1e6;
```

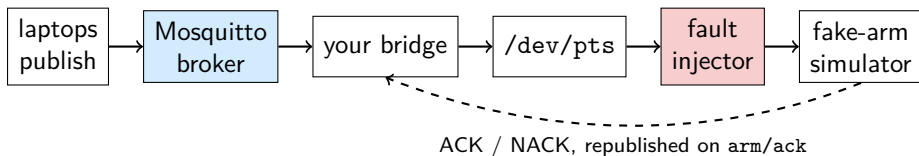
Never use the wall clock.

CLOCK_MONOTONIC cannot jump backwards; the wall clock can.



Project Hands-on Preview: Build the Bridge

The Hands-on Chain, Extended



Yesterday's chain is still there.

The serial port, fault injector and simulator are untouched. Today bolts a broker and a bridge onto the front.

The bridge is the seed of Friday's gatekeeper.

Everything between the bus and the actuator will grow from this program.

Hands-on Tasks

Task 1: forward.

Complete the bridge skeleton: subscribe to `arm/cmd` at QoS 1, validate with your Day 1 deframer, forward valid frames to the serial port, republish the simulator's ACK/NACK on `arm/ack`.

Task 2: measure.

The `arm/estop` echo: publish with a timestamp payload, echo it back, record round-trip times at QoS 0, 1 and 2. At least 200 runs each; report median and worst case.

Task 3: stress.

Flood `arm/cmd` from two laptops at once and watch the simulator lag behind the publishers. Where is the queue? Keep your answer for Day 3.

Hands-on Tasks (Cont'd)

Pass condition.

A complete per-QoS latency table for arm/estop, and a bridge that never forwards an invalid frame, even while the fault injector is running.

Keep what you build.

Day 3 opens up the sockets underneath this bridge; Day 5 rebuilds its loop around epoll; Friday's capstone turns it into the gatekeeper.

Optional Hardware, Same Principle

For groups with the Arduino arm.

Point the bridge's serial path at `/dev/ttyUSB0`. Now a phone on the lab Wi-Fi publishing to `arm/cmd` moves a physical servo. Nothing else changes.

The platform parallel.

The HCR platform routes everything through one named bus access point (called *Hotaru*) precisely so that simulator and real robot are a configuration choice, not a code change. Your bridge just demonstrated the same principle at lab scale.

Day 2 Summary

Pub-sub decouples, for a price.

A broker frees senders from knowing receivers; the price is hops and queues on every path, including the e-stop's.

QoS is a menu of delivery contracts.

Certainty is bought with round trips; choose per topic, by what the message means.

Your Day 1 work paid for itself.

CRC rejects damage; SEQ defuses QoS 1 duplicates. Layers compose.

Tomorrow.

Beneath the broker: sockets, file descriptors, and what happens when one slow client blocks everyone else.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template